HACKATHON SYSTEM ARCHITECTURE

x402 Red/Blue
Two-Laptop Security Demo

Detailed architecture, threat model, guard design, attack library, evaluation plan and live demonstration workflow

Architecture baseline: 31 October 2026

Core concept: RED operates a hostile merchant and facilitator; BLUE runs the payment agent protected by a nine-layer guard; a neutral mock chain independently determines whether funds moved.

Document basis

This document explains the supplied final architecture. It preserves the design's terminology, attack IDs, layer IDs, ports, implementation choices, verification status and stated risks. Items marked as predictions or not yet verified remain explicitly qualified rather than being presented as proven results.

RED TEAM • BLUE TEAM • NEUTRAL CHAIN • MEASURED SECURITY

x402 Red/Blue: Two-Laptop Demo Architecture

Oct 31, 2026 · @Sidakk

Summary

Laptop RED plays a hostile merchant and a lying facilitator; laptop BLUE runs a payment agent whose wallet is wrapped by a nine-layer guard; a neutral mock chain, not either side's logs, decides who lost money.

The demo story is a measured before and after on the same 16 attacks and the same seeds: the unguarded agent loses funds, the repo as shipped (Blue v0) still leaks, our hardened guard (Blue v1) loses nothing and blocks zero honest payments.

Three design decisions carry the whole architecture:

1. **Enforce at the signature, not only at the fetch.** x402-fetch hands the wallet one object, `walletClient.signTypedData(data)`, that holds the token contract, chain, recipient, amount and validity window. Blue v1 wraps the wallet so that call is checked against a pinned quote. Nothing a merchant, the SDK or our own code does earlier can change what gets signed.
2. **The chain is the judge.** Funds lost = balance that reached an attacker address on the mock chain. The guard's own decision log is a claim; the chain is the evidence, and the dashboard flags any disagreement.
3. **Allowlist, pin, reserve.** Replace the repo's blocklist, USD floats and after-the-fact ledger with an exact-host allowlist, atomic integer amounts, and a budget that is reserved before signing and released only when the chain confirms.

I ran the repo's real `PolicyEngine` against its shipped `config/policy.json` to confirm the gaps below are real, not guesses: a trailing-dot, subdomain or port variant of the blocked host all pass; any non-empty `approvalRef` such as `"x"` unlocks an above-threshold spend; and 20 checks fired before any spend was recorded approved $1.80 against a $1.00 daily budget.

Repo audit: what the code really does

The repo is about 1,300 lines of TypeScript in seven files; its value is a policy engine and a fetch wrapper, and everything we need to add sits in front of or around them. I read all of `src/` plus the x402 and x402-fetch 1.2.0 packages it depends on.

`src/wrap.ts` (141 lines)	`wrapPayerFetch` calls x402-fetch's `wrapFetchWithPayment` with a custom `select` callback that runs `engine.check()` before signing.	The one integration point. Blue v1 extends it.
`src/policy.ts` (319 lines)	`PolicyEngine`: daily, per-request, per-merchant and per-rail caps, host allow and block lists, network and rail lists. The ledger is a JSON file, loaded once in the constructor.	Reused as the budget skeleton, but its inputs and storage change.

`src/payments.ts` (387 lines)	`paywall()` Express middleware: builds `accepts[]`, verifies and settles through a facilitator URL.	Our honest merchant. Hostile mode wraps it. `FACILITATOR_URL` is an env var, so a mock facilitator drops in.
`src/server.ts` (150 lines)	Daemon on port 4032: paid `POST /policy-check`, admin `/ledger` and `/record-spend`.	Not on the guard's critical path in v1.
`src/sign.ts` (45 lines)	HMAC-SHA256 over canonical JSON; falls back to a public dev secret.	Reused for hashing the pinned quote. Verdict signatures stay advisory.

How a payment flows today. `wrapFetchWithPayment` fetches, gets a 402, parses `accepts` with the SDK's zod schema, calls our `select`, rejects anything above `maxValue`, calls `createPaymentHeader`, retries once. Our `select` runs `engine.check()` on `{merchant, amountUsd, resource, network, rail}` and returns the chosen requirement. Spend is recorded only after the retry returns 200 with an `x-payment-response` header.

What the SDK signs, and who controls it. For the EVM rail, x402 builds an EIP-3009 `TransferWithAuthorization` with `to = payTo`, `value = maxAmountRequired`, `validBefore = now + maxTimeoutSeconds`, and an EIP-712 domain whose `verifyingContract` is `asset` and whose name and version come from `extra`. Every one of those fields comes from the merchant's 402. The SDK's schema checks only that `payTo` and `asset` look like addresses and that the amount is a non-negative integer; I confirmed it accepts a made-up token address and a 9,999,999-second validity window.

Correction to the WalletShield document. It lists a TOCTOU gap between `select()` and signing. In 1.2.0 the object `select` returns goes straight into `createPaymentHeader` with no await in between, so that gap does not exist inside one call. The real pinning problem is different: nothing ties what the guard approved to what the wallet signs if the SDK changes, and nothing ties one signature to one intended purchase. Blue v1 closes that at the wallet (layer L4), which is stronger than an `IntentToken` handed around in memory.

Other facts that shape the design. The default `config/policy.json` has an empty `allowedMerchants`, so any host is payable up to $0.10 per request and $1.00 per day. Solana support is bundled; Blue v1 sets `allowedRails` to EVM only to remove half the attack surface. Passing a plain policy object to `wrapPayerFetch` creates an in-memory ledger that resets with every wrapper instance. `.env.example` ships with a real suite payee address pre-filled, so every demo `.env` must set its own.

Gap register: 18 gaps, each with evidence, an attack and a fix

The teammates' list had 15 gaps taken from the repo summary; reading the code confirms 13 of them (G10 and G13 stay unconfirmed) and adds 3 (G15, G16, G18). Each row maps to one attack, one layer and one regression test. Evidence tags: **Ran** = I executed the repo's own code, **Code** = read directly in the source, **SDK** = read in x402 1.2.0, **Doc** = from the teammates' PDF, still to confirm.

G1	The policy never sees the payee. `SpendIntent` has no `payTo` field, and the allowlist is by host.	Code	A1, A2	L2
G2	Token and chain are never checked. The SDK accepts any address-shaped `asset`, and `extra` supplies the EIP-712 domain.	SDK, Ran	A5	L2, L4

G3	Overcharge is allowed anywhere under the cap. Default policy: any host, up to $0.10 a call.	Code	A3	L3
G4	Budget race. Spend is recorded only after settlement, so concurrent checks all pass: 20 checks approved $1.80 on a $1.00 budget.	Ran	A7	L5
G5	Host controls are weak. The merchant is the host of the 402's own `resource` field, read from `accepts[0]`. The blocklist is an exact match: `evil.example.`, `sub.evil.example` and `evil.example:8443` all pass.	Ran, Code	A10	L0, L2
G6	No pinning at the wallet. The signer receives merchant-chosen domain and window; `accepts[0]` sets the merchant while another entry can be selected.	SDK, Code	A8, A14	L4
G7	Replay, expiry and resource binding rely on the facilitator. `validBefore` is `now + maxTimeoutSeconds`, unbounded (9,999,999 accepted). The authorization names no URL.	SDK, Ran	A9, A13	L4
G8	File ledger is unsafe across processes. It loads once in the constructor and rewrites the whole file on every append.	Code	A7	L5
G9	Daemon verdict is advisory and the paid check adds latency. The in-process wrapper already withholds the signature.	Code	none	L4
G10	Hostile 402 text could steer an LLM agent.	Doc	A12	L1
G11	No loop or velocity detection; slow drains stay under every threshold.	Code	A11	L6
G12	Amount parsing. Policy uses `Number(x)/1e6` floats while the SDK uses `BigInt`. `""` signs a zero-value authorization; `"1e3"` and `"1000.0"` pass the schema but crash `BigInt`.	Ran	A4, A6	L1
G13	Protocol version and header confusion.	Doc	A15	L1
G14	Settlement is trusted from the merchant. Spend is recorded only if the merchant sends `x-payment-response`; a merchant can settle the payment itself, omit the header, and it is never counted.	Code	A16	L5, L7

G15	The approval gate is a string check. Any non-empty `approvalRef` passes (`"x"` did).	Ran	A18	L3
G16	Dev secrets. `SIGNING_SECRET` falls back to a public constant; `ADMIN_KEY` defaults to `dev-admin-key`.	Code	none	L8
G17	No tests, collector, dashboard or red team exist.	Code	all	build
G18	Retry double-pay. If the merchant takes the payment and returns 5xx, the agent retries and signs a second authorization; nothing tracks outstanding signatures.	Code	A17	L5

A17 (settle-and-fail) and A18 (approval bypass) are additions to the PDF's A1 to A16. A16 now covers both directions: success reported with no transfer, and a transfer with no receipt.

Two-laptop topology and network plan

Blue and Red each run three services on their own laptop, and the only traffic between them is what a real merchant and a real agent would exchange, plus a test-only control channel and the event feed.

[embedded content: two-laptop topology · 2 laptops, 6 services, 1 neutral chain]

Red's merchant answers the agent's request with a 402, Red's facilitator settles through the chain, and Red's console both triggers the agent and reports every attempt to the collector. Blue's guard sees only what arrives over the wire.

Network plan

1. **Primary link:** both laptops join one phone hotspot. Address each by its hotspot IP in two environment variables, `BLUE_URL` and `RED_URL`.
2. **Fallback link:** a direct USB-C Ethernet cable with static addresses, `192.168.77.1` for laptop A and `192.168.77.2` for laptop B. Test it before the event; do not discover it on stage.
3. **Last resort:** `npm run solo` runs both sides on one laptop over `localhost`.
4. **Ports:** open 4010, 4040 and 4050 on laptop A and 4020, 4030 and 4060 on laptop B. The agent's `/control` endpoint accepts only the Red laptop's address.
5. **Clocks:** the agent and the chain share laptop A's clock, which is why the chain lives there. Sync both laptops anyway, because the signing window is 30 seconds.
6. **No tunnel needed.** Add Cloudflare Tunnel only if judges must open the dashboard on their own phones.

System architecture and request flow

A payment can leave the flow at any of three checkpoints, and only the third can produce a signature, so a payment that fails an earlier check never reaches the wallet at all.

[embedded content: one payment through Blue v1 · 7 steps, 3 checkpoints, 1 block exit]

In steps 3 to 5 the first failed layer blocks the payment, records one reason code and stops. In step 6 the signed authorization is spent by whoever holds it, which is why L5 keeps the exposure reserved until step 7 sees the chain's own balance change.

Trust boundaries

- **Trusted:** the guard, its policy file, the wallet signer, all inside the agent's process; and the mock chain's ledger.
- **Untrusted:** the merchant, the facilitator's replies, and every field, header and line of text in a 402.
- **Test-only:** the control channel from the Red console to the agent and merchant. Agent code and the guard cannot read or use it.
- **Agent code has no policy power.** The daemon's signed verdicts stay advisory; the wrapper in the agent's process is what withholds the signature.

Tech stack

One language, TypeScript on Node, runs on both laptops, so the attack modules, the guard and the dashboard share one set of types and one event schema. Everything the repo already uses stays; the additions are small and each has a fallback.

Language and runtime	TypeScript 5.7 strict, Node 22 LTS, `tsx` for dev	The repo is strict TypeScript on Node 18 or later. Same toolchain, no new build.
Base code	`nirholas/x402-agent-wallet` (Apache-2.0), forked into an npm-workspaces monorepo	Keep the LICENSE and credit it in the README; confirm the licence terms with the organisers.
x402 SDK	`x402` and `x402-fetch`, pinned to exactly 1.2.0 (no caret)	The behaviour audited here is 1.2.0. A silent upgrade could change what gets signed.
Wallet and crypto	`viem` 2.x: `privateKeyToAccount`, `verifyTypedData`, `keccak256`; `node:crypto` SHA-256	Already a repo dependency. The wallet wrapper and the mock chain both use it.
Schema validation	`zod` with `.strict()` objects, direct dependency	Rejects unknown, duplicate and oversized fields in the 402 (L1). x402 already pulls it in.
Budget store	`better-sqlite3` in WAL mode, `BEGIN IMMEDIATE` transactions; fallback `node:sqlite`	Atomic reserve, commit and release across processes. Replaces the JSON file ledger.
Host handling	WHATWG `URL`, `node:url` `domainToASCII`	No extra library: exact-match ASCII allowlist makes homograph and punycode tricks fail by construction.
Honest merchant	The repo's own `paywall()` from `payments.ts`, behind a mutator layer	Reuses the real backend; hostile modes only edit the 402 before it is sent.
Mock facilitator	Express, implements `POST /verify`, `POST /settle` as `useFacilitator` calls them	Red controls it, so it can delay, fail or lie (A16, A17).
Mock chain	Express service with an in-memory EIP-3009 token: `viem.verifyTypedData`, nonce set, time window, balances. Stretch: Anvil with a state dump of the Base Sepolia USDC contract	Ground truth for funds lost. The default needs no internet at the venue.
Red tooling	`commander` CLI, `fast-check` for seeded property fuzzing	Every bypass replays from a seed and shrinks to a minimal case.

Learning attacker	TypeScript UCB1 bandit, then a (1+λ) evolution strategy. Stretch: Python 3.11 with Stable-Baselines3 PPO over an HTTP environment	Trains in minutes, no deep-learning stack. Drop the PPO first if time runs short.
Collector and events	Express, HTTP POST in, Server-Sent Events out, SQLite event store	One shared log for both laptops; SSE needs no WebSocket library.
Dashboard	React 18, Vite 5, Tailwind, Recharts, served by the collector	Two views from one build: Blue console and Red console.
Tests and CI	Vitest, GitHub Actions	Legitimate-traffic gate and the attack suite run on every merge.
Logging	`pino` JSON lines, policy digest on every line	Matches the PDF's rule that the digest is stamped into every log line.
Orchestration	npm workspaces, `concurrently`: `npm run blue` on laptop A, `npm run red` on laptop B, `npm run solo` for one laptop	One command per laptop on stage.
Network	Phone hotspot first; a direct USB-C Ethernet cable with static IPs as the fallback; `localhost` single-laptop mode last	Venue Wi-Fi often isolates clients. No tunnel is needed for two laptops.
Backup	Screen recording of a full run, exported results JSON, screenshots	Per the PDF's backup plan.

Blue v1: the guard, layer by layer

Blue v1 is a library inside the agent's process that touches a payment at three points: a guarded fetch (L0 to L2), a guarded selector (L3, L5, L6), and a guarded wallet that is the last thing before any signature (L4). After the response, a reconciler (L7) checks the chain, and every step writes an event (L8). Any exception inside the guard is a block with reason `GUARD_ERROR`; there is no default-allow path.

L0 Transport	Guarded fetch	The URL's exact origin (scheme, ASCII host, port) is not allowlisted; the URL has userinfo; any redirect appears (`redirect: "manual"`, no payment is ever carried across one); an HTTPS origin is reached over HTTP. A trailing dot is stripped before the exact match.	`REDIRECT_HOST`, `ORIGIN_NOT_ALLOWED`
L1 Parse	Guarded fetch, on every 402	Body over 8 KB; duplicate or unknown keys (`zod.strict()`); `x402Version` not exactly 1; scheme not `exact`; amount not matching `^(0|[1-9][0-9]{0,18})$` or equal to zero; more than 4 options. The guard then rebuilds a minimal 402 with protocol fields only and hands that to the SDK, so `description` and `outputSchema` text never reaches an LLM (A12).	`PARSE_REJECT`
L2 Identity	Guarded fetch	The endpoint (origin, method, path) is not in the registry; `payTo`, `asset` (contract), chain ID, or the `extra` name and version differ from the registry entry. Addresses compare as checksummed values.	`ENDPOINT_UNKNOWN`, `PAYTO_MISMATCH`, `ASSET_NOT_ALLOWED`, `NETWORK_NOT_ALLOWED`
L3 Price	Guarded selector	Amount outside the endpoint's price band; amount above an earlier quote for the same method and URL; amount at or above the approval threshold without a one-time approval token bound to the pinned hash.	`PRICE_OUT_OF_BAND`, `PRICE_RISE`, `APPROVAL_REQUIRED`
L4 Pin and signer firewall	Guarded wallet	The typed data differs from the pinned quote in any field; the validity window is over 30 seconds; the pin was already used; no pin exists; the wallet is asked to sign anything but `TransferWithAuthorization`.	`QUOTE_CHANGED`, `WINDOW_TOO_LONG`, `PIN_CONSUMED`, `SIGN_UNPINNED`, `SIGN_FORBIDDEN`

L5 Budget	Selector (reserve), reconciler (commit, release)	Spent plus reserved plus this amount exceeds a daily, per-merchant or per-call cap; the same intent already has an outstanding authorization.	`BUDGET_RESERVED`, `BUDGET_EXHAUSTED`, `DUPLICATE_INTENT`
L6 Behaviour	Selector	Repeated 402s on one resource with no confirmed delivery; payments per minute above the velocity limit. The score can tighten limits or call for approval, never loosen a rule.	`LOOP_DETECTED`, `VELOCITY`
L7 Post-payment	Reconciler	The agent's balance change on the chain differs from the pinned amount, or the receipt names a different payee, asset or amount. Raises an alert and holds the exposure.	`SETTLEMENT_MISMATCH`
L8 Decision log	Every layer	Never blocks. Writes ALLOW or BLOCK, reason, pin hash, policy digest and trace ID locally and to the collector; if the collector is down it buffers.	none

Why L4 is the centrepiece. The SDK builds the EIP-712 domain and the validity window from merchant-supplied fields, then calls `walletClient.signTypedData`. At that call the guard sees the final bytes: contract, chain, recipient, amount, window. It compares them with the hash it pinned during L0 to L3. A bug in the SDK, a field the guard failed to parse, or an attack that mutates terms between steps all end in the same place: `QUOTE_CHANGED`, and no signature exists. The guarded fetch also clamps `maxTimeoutSeconds` to 30 in the rebuilt 402, so the SDK itself builds a short window and the wallet check only has to enforce it.

Why L5 holds exposure until the chain says otherwise. A signed authorization is spendable by the merchant until its `validBefore`. So a reservation moves from RESERVED to SIGNED to CONFIRMED (chain balance fell by the pinned amount), RELEASED (the payment was refused before signing), or HELD until the window expires and the chain shows no debit. The merchant's `x-payment-response` header is logged but never decides anything. This closes G14 and G18.

Where it plugs into the repo

- `src/wrap.ts`: `wrapPayerFetch` gains a `guard` option. It passes `guardFetch(baseFetch)` and `guardWallet(account)` into `wrapFetchWithPayment`, replaces `select` with the guarded selector, drops the `Number(x)/1e6` conversion, and stops recording spend from the merchant's header.
- `src/guard/` (new): `transport.ts`, `parse402.ts`, `registry.ts`, `price.ts`, `pin.ts`, `walletGuard.ts`, `budget.ts`, `behaviour.ts`, `reconcile.ts`, `events.ts`, `reasons.ts`.
- `src/policy.ts`: keep the cap logic, move amounts to `bigint` atomic units and storage to SQLite; USD exists only for display.
- `src/sign.ts`: reuse `canonicalize` for the pin hash; refuse to start if `SIGNING_SECRET` or `ADMIN_KEY` is still the default (G16).
- `config/policy.v1.json` (new): the endpoint registry below.

Policy file, as data

```
{
  "version": 1,
  "rails": ["evm"],
  "caps": { "dailyAtomic": "1000000", "perCallAtomic": "100000", "approvalAtomic": "50000", "velocityPerMin": 20 },
  "endpoints": [{
    "origin": "http://<RED_IP>:4020",
    "method": "GET",
    "path": "/paid/report",
    "payTo": "0x<merchant wallet>",
    "asset": "<USDC address from x402 for base-sepolia>",
    "chainId": 84532,
    "assetName": "<extra.name from x402>",
```

```
    "assetVersion": "<extra.version from x402>",
    "price": { "minAtomic": "1000", "maxAtomic": "10000" },
    "maxWindowSeconds": 30
  }]
}
```

The demo origin is plain HTTP because both laptops sit on a LAN. The allowlist stores the full origin, so an HTTPS entry still rejects its HTTP twin, which is exactly what attack A10 tests.

Sketch of the wallet guard

```
export function guardWallet(account: LocalAccount, pins: PinStore, log: Events): LocalAccount {
  const refuse = (code: ReasonCode, detail: unknown): never => {
    log.block(code, detail);
    throw new GuardBlock(code);
  };
  return {
    ...account,
    signMessage: () => refuse("SIGN_FORBIDDEN", "signMessage"),
    signTransaction: () => refuse("SIGN_FORBIDDEN", "signTransaction"),
    async signTypedData(data) {
      if (data.primaryType !== "TransferWithAuthorization") refuse("SIGN_FORBIDDEN", data.primaryType);
      const seen = pinHashFromTypedData(data, account.address, Date.now());
      const pin = pins.take(seen.hash);               // single use
      if (!pin) refuse(pins.has(seen.hash) ? "PIN_CONSUMED" : "SIGN_UNPINNED", seen);
      if (seen.windowSeconds > 30) refuse("WINDOW_TOO_LONG", seen);
      const sig = await account.signTypedData(data);
      log.allow(pin);
      return sig;
    },
  };
}
```

This is a design sketch to be finished against the real types in the Phase 0 spike, including a check that x402-fetch's account detection still accepts the wrapped object.

Red: attack library, fuzzer and learner

Red runs on laptop B as a hostile merchant plus a lying facilitator, driven by a CLI and a console. It has three tools: 18 scripted attacks with fixed seeds, a seeded mutation fuzzer, and a learner that searches the same space and reports bypasses. A bypass counts only if the **mock chain** shows money reaching a wrong party or exceeding the intended price, or if the guard's own decision disagrees with the registry.

The Blue v0 column is a prediction from reading the code unless marked **Ran**; Phase 1 replaces every prediction with a measurement.

A1	Payee swap in one request	The registry's payee is A; the 402's `payTo` is attacker address B.	Leaks (policy never sees `payTo`)	L2 `PAYTO_MISMATCH`

A2	Payee swap on request N	N-1 honest 402s, then the attacker address.	Leaks	L2 `PAYTO_MISMATCH`
A3	Price creep	Amount rises a little each call, staying under the cap.	Leaks (any price up to $0.10)	L3 `PRICE_RISE`, `PRICE_OUT_OF_BAND`
A4	Drain	One amount equal to the whole balance.	Blocked (per-request cap, SDK `maxValue`)	L1 and L3
A5	Fake asset or chain	Same symbol, different contract address or chain ID; `extra` carries a fake EIP-712 domain.	Leaks (asset never checked)	L2 `ASSET_NOT_ALLOWED`
A6	Amount parsing	`""`, `"1e3"`, `"1000.0"`, `"0x10"`, leading zeros, a 30-digit number.	Mixed: fails safe by accident	L1 `PARSE_REJECT`
A7	Parallel burst	100 concurrent requests that each pass before any settles.	Leaks (**Ran**: 20 checks approved $1.80 on $1.00)	L5 `BUDGET_RESERVED`
A8	Compromised dependency	Test hook swaps the requirement object between approval and signing, simulating a malicious SDK update.	Leaks (nothing checks at the wallet)	L4 `QUOTE_CHANGED`
A9	Long-lived authorization	`maxTimeoutSeconds` of 9,999,999; the merchant settles after the agent's day has rolled over.	Leaks	L4 `WINDOW_TOO_LONG`
A10	Redirect and lookalike host	302 to another host; `evil.example.`, `sub.evil.example`, `evil.example:8443`, a homograph name, HTTP twin of an HTTPS origin.	Leaks (**Ran** for the three blocklist variants)	L0 `REDIRECT_HOST`, `ORIGIN_NOT_ALLOWED`
A11	Slow drain loop	Repeated valid-looking 402s on one resource, each under every threshold.	Leaks up to the daily cap	L6 `LOOP_DETECTED`, `VELOCITY`
A12	Prompt injection	`description` and body text telling the agent to pay and ignore limits.	Text reaches the agent unfiltered	L1 strips it; decision never reads it
A13	Binding hygiene	Agent code sends a signed payment to a URL other than the pinned one. No funds move, but a bearer authorization leaves its context.	Not prevented	L0 and L4 pin includes URL
A14	Multi-option confusion	`accepts[0]` shows an allowlisted `resource`; the entry the client selects carries the attacker's `payTo`.	Leaks (merchant read from `accepts[0]`)	L1 collapses to one entry, L2 checks it
A15	Header and version confusion	Old and new header names, duplicate headers, mixed `x402Version`.	Unknown; measure	L1 `PARSE_REJECT`
A16	Settlement lies	Facilitator reports success with no transfer; or transfers with no `x-payment-response`.	Leaks (second form is never recorded)	L7 `SETTLEMENT_MISMATCH`
A17	Settle-and-fail	Merchant settles, returns 500; the agent retries and signs a second authorization.	Leaks (no record of the first)	L5 `DUPLICATE_INTENT`
A18	Approval bypass	Supply `approvalRef: "x"` on the daemon's `/policy-check` path.	Leaks on the daemon (**Ran**); the wrapper never sends one	L3 one-time token

Fuzzer

Takes a valid 402 and mutates one field at a time, then pairs: amount, `payTo`, `asset`, network, `maxTimeoutSeconds`, `extra`, resource URL, scheme, version, headers, key order, encodings. `fast-check` generates the cases from a seed and shrinks any bypass to a minimal 402. The oracle is the registry: any ALLOW whose terms differ from the registry entry is a bypass, and any chain loss is a bypass whatever the guard said. The same run goes at the naive agent, Blue v0 and Blue v1.

Learner

1. Actions are an attack ID plus parameters: amount multiplier, address variant, host variant, delay, concurrency.
2. A UCB1 bandit picks the attack and parameter bucket; a (1+λ) evolution strategy then mutates the best parameter vector. This trains in minutes with no deep-learning stack.
3. Reward comes from the chain, not from the guard: large when funds reach a wrong party, small and negative when blocked, a bonus for a bypass no earlier run found.
4. Each laptop worker uses its own seed and posts every attempt to the collector; one process holds the policy.
5. A person replays every reported bypass against the real protocol flow before it counts, because a learner will exploit a bug in our own mock.
6. PPO with Stable-Baselines3 is a stretch goal and the first thing dropped when time is short.

The result on screen is a learning curve of bypass rate per episode, logged on every run: high against Blue v0, falling to zero against Blue v1 as each finding is patched.

Conduct and reach

Red attacks only our own stack and any target the organisers provide, with test funds only, a per-worker rate limit and a kill switch on the dashboard. For another team's guard, Red speaks whatever interface the organisers define; if they define none, a small adapter posts a 402 to `POST <target>/guard/check` and reads `{decision, reason}`.

Mock stack, collector and dashboard

Six services make the demo run; each is small, and each sits on the side of the trust boundary where its real-world counterpart would be.

Guarded agent	A (Blue)	4010	Naive agent and guarded agent in one process, switchable at runtime between naive, Blue v0 and Blue v1. Holds the test wallet. Has a test-only `/control` endpoint, reachable only from the Red laptop's address, never from agent code.
Mock chain	A (Blue)	4040	Neutral ledger of an EIP-3009 token. Verifies the signature with `viem.verifyTypedData`, checks the nonce, the time window and the balance, then moves funds. Read-only balances for everyone; transfers only through `settle`.
Collector and Blue console	A (Blue)	4050	Stores every event in SQLite, streams them by SSE, serves the dashboard.
Hostile merchant	B (Red)	4020	Wraps the repo's `paywall()`. Honest by default; any of A1 to A18 can be switched on per request through the control channel.

Mock facilitator	B (Red)	4030	Implements `POST /verify` and `POST /settle`, the two calls `useFacilitator` makes. Can delay, fail or lie. Settles by calling the mock chain, like a real facilitator with a gas key.
Red console	B (Red)	4060	Attack launcher, seed picker, live bypass counter, learning curve.

Why the chain lives on Blue's laptop but is not Blue's code. Judges should be able to ask who decides "funds lost". The answer is the mock chain: a separate process with its own database that neither the guard nor the attacker can edit, only transact against, as with a real chain. The facilitator on laptop B is what Red controls; the chain on laptop A is what both must obey.

Event schema

Every attempt, from either laptop, posts one record to the collector. This extends the PDF's schema with the pin hash, the layer, and the chain's own verdict so the dashboard can show disagreement.

```
{
  "run_id": "r-0042",
  "worker": "red-laptop",
  "attack": "A7",
  "seed": 1187,
  "target": "blue-v1",
  "decision": "BLOCK",
  "reason": "BUDGET_RESERVED",
  "layer": "L5",
  "pin_hash": "sha256:...",
  "amount_atomic": "250000",
  "chain_delta_atomic": "0",
  "funds_lost_atomic": "0",
  "latency_ms": 3.2,
  "policy_digest": "sha256:...",
  "ts": "2026-10-31T10:15:00Z"
}
```

`funds_lost_atomic` is filled by the mock chain after settlement, never by the guard. If `decision` is BLOCK and `chain_delta_atomic` is not zero, the dashboard raises a red "guard and chain disagree" banner.

Dashboard views

- **Blue console (laptop A, the judges' screen):** one large "funds lost" number; a v0 against v1 split view replaying the same seeds; live decision stream with reason codes; budget gauge showing spent, reserved and free; the policy digest; an arm and disarm switch; the one-time approval button.
- **Red console (laptop B):** attack launcher (attack, seed, count); a matrix of attacks against targets coloured by outcome; a bypass counter; the learning curve; the kill switch that halts every worker.
- **Results export:** one JSON file regenerates every table on every slide. Numbers not in the logs do not go on a slide.

Demo script and backup plan

The five-minute demo has one story: the agent loses money, the stock guard still leaks, our guard holds, and a judge tries to break it. Laptop A (Blue) drives the projector; laptop B (Red) sits beside it with the attacker's console visible, so judges see attacker and defender side by side. Adjust the timings once the organisers give the slot length.

1. The problem	0:00 to 0:30	Console open, attacks idle.	Dashboard live, agent wallet funded, funds lost = 0.
2. Naive agent	0:30 to 1:15	Launches the full suite at the naive agent.	Funds lost climbs live, tagged by attack.
3. Blue v0	1:15 to 2:15	Replays the same seeds at the repo as shipped.	Most attacks blocked; payee swap, parallel burst, host variants and price creep get through. Funds lost is not zero.
4. Blue v1	2:15 to 3:15	Replays the identical seeds.	Every attack blocked with a reason code each; funds lost = 0; false-block counter at 0 of 200 honest payments; p95 guard latency.
5. Learner	3:15 to 4:00	Shows the learning curve from real logs.	Bypass rate high against v0; against v1 it falls to zero.
6. Judge's turn	4:00 to 5:00	A judge edits `amount` or `payTo` in a hand-craft form and sends it.	The verdict and its reason code appear; the chain shows no transfer.

What makes it land. A split screen with v0 on one side and v1 on the other, running the same seeds. One big number, funds lost, always visible. A reason code on every block. Each console shows its operator's name next to the attack class they are running.

Pre-flight checklist (T minus 15 minutes)

1. Both laptops on the phone hotspot; `curl` each other's health endpoints. Cable fallback tested with static IPs.
2. Clocks synced on both machines. The signature window is 30 seconds, so a drifting clock turns honest payments into false blocks.
3. `npm run reset` on laptop A: fresh chain, wallet funded, budget empty, policy digest shown on screen.
4. The 200-payment honest run passes with zero blocks.
5. Demo build is the frozen commit; the recorded run and exported results file are open in another window.

Backup plan

- A recorded full run of beats 2 to 5, ready to play.
- Exported results JSON and screenshots of the final tables.
- `npm run solo` runs all workers and both consoles on one laptop over `localhost`.

- Hotspot and cable both tested before the slot; venue Wi-Fi is never relied on.
- Nothing in the demo needs a public chain, a faucet or the internet.

Build plan, owners and repo layout

Five owners keep the PDF's split; two of them live on each laptop and the environment owner works across both. Everything shares one event schema and one reason-code file, agreed in Phase 0, so every piece has something real to test against by the end of Phase 1.

Person 1: Environment	Both	Hostile merchant on `paywall()`, mock facilitator, mock chain, naive agent, `solo` mode, reset script	none
Person 2: Blue v1	A	Layers L0 to L8, policy file, wallet guard, SQLite budget	Person 1 for the mock
Person 3: Red library	B	Attack modules A1 to A18 with seeds and pass or fail checks, fuzzer	Person 1 for the mock
Person 4: Dashboard and logging	A	Collector, event schema, both consoles, v0 against v1 toggle, kill switch	Event schema from Phase 0
Person 5: Learner and evaluation	B	Bandit and evolution search, metrics tables, regression runner, results export	Person 3 for modules

Phases

0. Contract and spike	Event schema, reason codes, policy file format, attack IDs on one page. Read the contest spec. Run the five spikes below.	Everyone has read the page; all five spikes pass or have a named fallback.
1. First blood	Mock stack running; naive agent loses money to A1 and A4; Blue v0 runs as the unmodified repo.	A recorded run where v0 blocks some attacks and leaks others, with the chain's numbers.
2. Harden and widen	Blue v1 layers added one at a time, each paired with its attack and a test. All 18 attacks done. Dashboard live.	The full suite shows v1 blocking everything with zero false blocks over 200 honest payments.
3. Automate	Fuzzer and learner run from laptop B; differential table for v0, v1 and any other guard allowed.	A learning curve generated from real logs.
4. Polish and rehearse	Demo script, backup recording, judge's-turn practice.	Two full rehearsals on the hotspot.

Freeze points as fractions of the time available: about 70% feature freeze, about 85% demo build frozen and recording made, about 90% final rehearsal with both laptops on the hotspot. Every merge runs the 200-payment honest set; a rise in false blocks fails the build.

Phase 0 spikes: settle these before building on them

1. A wrapped viem account (`{...account, signTypedData}`) is still accepted by x402-fetch's wallet detection and produces a payment the mock facilitator verifies.
2. A guarded base fetch can return a rebuilt 402 `Response` and the SDK signs from it.
3. The mock facilitator satisfies `useFacilitator`, which posts `{x402Version, paymentPayload, paymentRequirements}` to `/verify` and `/settle`, and the repo's `paywall()` works unchanged against it through `FACILITATOR_URL`.
4. `better-sqlite3` installs on both laptops; if it does not, switch to `node:sqlite`.
5. The mock chain's EIP-3009 check agrees with the SDK's signature: a payment the SDK signs verifies; a tampered one does not.

Repository layout

```
x402-redblue/
  packages/
    guard/        L0-L8, wallet guard, SQLite budget      (fork of x402-agent-wallet/src)
    mock/         merchant, facilitator, chain, naive agent
    red/          attacks A1-A18, fuzzer, learner, CLI
    collector/    events, SSE, results export, serves dashboard
    dashboard/    React + Vite, Blue and Red consoles
  config/
    policy.v1.json  reasons.ts  events.schema.json
  tests/
    honest-200.test.ts   attacks/*.test.ts   regressions/seed-*.test.ts
  scripts/   reset.ts   solo.ts   export-results.ts
```

Risks and questions to settle first

The biggest risk is building on assumptions the contest spec contradicts, so the first task is to answer the questions below with the organisers.

Ask the organisers

1. What interface must the guard expose (language, API, harness), and does it have to run in the agent's process?
2. How are Red and Blue scored, and is the server or facilitator side in scope?
3. Which networks, assets and protocol version does the harness use? This design assumes x402 version 1, EVM only, and the `X-PAYMENT` header.
4. Which targets may we attack, through what channel, at what rate?
5. May we build on the open-source `x402-agent-wallet` repo, and what credit does its Apache-2.0 licence require?
6. How long is the event, and how long is the demo slot?

What is verified and what is not

- **Verified by running code:** the three blocklist bypasses, the `approvalRef` bypass, and the $1.80-on-$1.00 budget race against the repo's own `PolicyEngine` and shipped policy; and the SDK's acceptance of odd amounts, a made-up asset address and a huge validity window.

- **Verified by reading source:** how x402-fetch 1.2.0 selects, signs and retries; what the EIP-712 domain and window are built from; that `merchantOf` reads the merchant-supplied `resource`; that spend is recorded only on the merchant's header.
- **Not yet verified:** every "Blue v0 leaks" prediction other than those above; the wallet-wrapper sketch against real viem types; the mock facilitator's fit with `useFacilitator`; the exact `extra` name and version for the demo token. Phase 0 and Phase 1 settle these.
- **Out of scope by design:** the Solana rail (disabled in the Blue v1 policy), server-side defences unless the contest scores them, and anything on a public chain.

Risks and fallbacks

Venue network isolates the two laptops	Phone hotspot, then the direct Ethernet cable with static IPs, then `npm run solo` with the recorded run.
Guard blocks an honest payment and loses points	The 200-payment honest run gates every merge; clock sync is on the pre-flight list because of the 30-second window.
The wrapped wallet is not accepted by x402-fetch	Spike 1 finds this early; fallback is to fork the signing call in a pinned copy of the SDK, keeping the same pin check.
`better-sqlite3` fails to build	Use `node:sqlite`; the budget interface is the same.
Learner exploits a bug in our own mock	A person verifies every bypass against the real flow before it counts.
Time runs short	Drop in this order: PPO, the learner, anvil mode, server-side defences. Keep the scripted attacks, Blue v1, the mock chain and the dashboard.
Judges know existing open-source guards	Show the differential table and the bypasses found in v0, each with its seed.
The repo's code differs from what the PDF assumed	Already re-checked against the source: see the gap register.